

Python Operators and Input

Operators are symbols that perform operations on variables and values.

1. Arithmetic Operators:

Used to perform basic mathematical operations.

OPERATOR	NAME	EXAMPLE	OUTPUT
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2	2.5
//	Floor Division	5 // 2	2
%	Modulus	5 % 2	1
**	Exponentiation	2 ** 3	8

Code:

```
a = 10
b = 3
print("Addition:", a + b)    # 13
print("Subtraction:", a - b) # 7
print("Multiplication:", a * b) # 30
print("Division:", a / b)    # 3.333...
print("Floor Division:", a // b) # 3
print("Modulus:", a % b)    # 1
```

```
print("Exponentiation:", a ** b) # 1000
```

2. Comparison Operators

Used to compare two values and return a boolean (True or False).

OPERATOR	DESCRIPTION	EXAMPLE	OUTPUT
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater or equal	5 >= 5	True
<=	Less or equal	3 <= 5	True

Code :

```
x = 5
y = 7
print("x == y:", x == y) # False
print("x != y:", x != y) # True
print("x > y:", x > y) # False
print("x < y:", x < y) # True
print("x >= y:", x >= y) # False
print("x <= y:", x <= y) # True
```

3. Logical Operators

Used to combine conditional statements.

OPERATOR	DESCRIPTION	EXAMPLE	OUTPUT
AND	True if both are true	True and False	False
OR	True if at least one is true	True or False	True
NOT	Inverts the result	not True	False

Code:

```
a = 5  
  
print (a > 3 and a < 10) # True  
  
print (a < 3 or a == 5) # True  
  
print (not(a == 5))    # False
```

4. Assignment Operators:

Theory:

Used to assign or update values of variables.

Operator	Description	Example
=	Assign	x = 5
+=	Add and assign	x += 3
-=	Subtract and assign	x -= 3
*=	Multiply and assign	x *= 3
/=	Divide and assign	x /= 2
//=	Floor divide assign	x //= 2

%=	Modulus assign	x %= 2
**=	Power assign	x **= 2

Code :

```

x = 5
x += 3 # 8
x -= 2 # 6
x *= 2 # 12
x /= 3 # 4.0
x //= 2 # 2.0
x %= 2 # 0.0
x = 4
x **= 2 # 16
print ("Final x:", x)

```

5. Identity Operators Theory:

Used to compare memory locations of two objects.

OPERATOR	DESCRIPTION	EXAMPLE
IS	True if same object	a is b
IS NOT	True if not same object	a is not b

Code: a = [1, 2] , b = a , c = [1, 2]

```
print (a is b)    # True
```

```
print (a is c)    # False
```

```
print(a is not c) # True
```

Membership Operators:

Used to test if a value exists in a sequence (string, list, tuple, etc).

OPERATOR	DESCRIPTION	EXAMPLE
IN	True if value exists	'a' in 'apple'
NOT IN	True if value not exists	'x' not in 'box'

Code:

```
name = "python"
```

```
print('y' in name) # True
```

```
print('z' not in name) # True
```

7. Bitwise Operators

Work at the **bit level** of numbers (binary representation).

Operator	Name	Example	Binary Logic
&	AND	5 & 3 → 1	
	OR	5 3 → 7	
^	XOR	5 ^ 3 → 6	
~	NOT	~5 → -6	
<<	Left Shift	5 << 1 → 10	
>>	Right Shift	5 >> 1 → 2	

Code:

```
a = 5 # 0101
```

```
b = 3 # 0011
```

```
print("AND:", a & b) # 0001 -> 1
```

```

print("OR:", a | b)    # 0111 -> 7
print("XOR:", a ^ b)   # 0110 -> 6
print("NOT:", ~a)      # -(5+1) -> -6
print("Left Shift:", a << 1) # 1010 -> 10
print("Right Shift:", a >> 1) # 0010 -> 2

```

Chained Conditions

```
x = 10
```

```
print(5 < x < 15) # True, same as: (5 < x) and (x < 15)
```

Summary Table

CATEGORY	OPERATORS USED
ARITHMETIC	+, -, *, /, %, //, **
COMPARISON	==, !=, >, <, >=, <=
LOGICAL	and, or, not
ASSIGNMENT	=, +=, -=, *=, /=, //=, %=, **=
IDENTITY	is, is not
MEMBERSHIP	in, not in
BITWISE	&,

Understanding input () in Python

The input () function is used to take **user input** from the keyboard.

Syntax:

```

variable = input("Enter something: ")
name = input("Enter your name: ")

print("Hello", name)

```

Practice Set:

Question 1: Arithmetic Operators

Write a Python program to perform and display all arithmetic operations on a = 12 and b = 5.

Include: +, -, *, /, //, %, **.

Question 2: Comparison Operators

Take two numbers as input and check:

- Are they equal?
- Is the first greater?
- Is the second less or equal to the first?

Use all six comparison operators in the code.

Question 3: Logical Operators

Let x = 15.

Check and print the result of the following:

- x is greater than 10 **and** less than 20
- x is equal to 15 **or** equal to 10
- Not (x is 15)

Question 4: Assignment Operators

Start with x = 8.

Apply the following operations step-by-step and print the updated value of x each time:

- += 2
- *= 3
- /= 2
- //= 2
- %= by 3
- **= by 2

Question 6: Membership Operators

```
text = "python"  
items = [1, 2, 3, 4]
```

Check:

- 't' in text
- 'z' not in text
- 2 in items
- 5 not in items

Question 7: Bitwise Operators & Chained Condition

Let a = 6, b = 3, x = 12.

Perform:

- a & b, a | b, a ^ b, ~a, a << 1, a >> 1

Also check:

- Is x between 10 and 15 using chained condition?